

Revisão da Primeira Prova Bimestral

Uma leitura integrada dos conceitos centrais da aula.

Visão geral

Este capítulo parte de razões para estudar conceitos de LP e avança até escopo, tempo de vida e constantes nomeadas. No percurso, articula critérios de avaliação de linguagem, confiabilidade e verificação de tipos, custo de uma linguagem, influências no projeto de linguagens e categorias de linguagens. Em vez de tratar cada termo como uma definição isolada, a discussão mostra como os conceitos se conectam nas escolhas feitas por quem projeta, avalia ou utiliza linguagens de programação.

Razões para estudar conceitos de LP: critérios e conexões

O ponto de partida é **Razões para estudar conceitos de LP**: O estudo dos conceitos de linguagens de programação amplia a expressão de ideias, fundamenta a escolha de linguagens, facilita a aprendizagem de novas linguagens, melhora o uso das já conhecidas e ajuda a compreender a implementação. A discussão avança para **Críticos de Avaliação de Linguagem**: Legibilidade, capacidade de escrita, confiabilidade e custo são critérios usados para comparar a adequação de uma linguagem, sem pressupor que uma única linguagem maximize todos eles. Outro elemento dessa análise é **Legibilidade e ortogonalidade**: Legibilidade envolve a facilidade de compreender programas. A ortogonalidade contribui para ela ao permitir que componentes básicos sejam combinados com poucas restrições e exceções. Na prática, a análise passa por **Capacidade de escrita**: É a facilidade com que uma linguagem pode ser usada para criar programas destinados a um domínio de problema específico. Ao mesmo tempo, o quadro se completa com **Confiabilidade e verificação de tipos**: Confiabilidade é o comportamento de acordo com as especificações. A verificação de tipos contribui para esse critério ao detectar usos incompatíveis, preferencialmente durante a compilação para reduzir o custo de correção. Por fim, esta parte se encerra com **Custo de uma linguagem**: O custo inclui treinamento, aprendizagem, escrita de programas, implementação e execução. Decisões como verificações em tempo de execução podem elevar a confiabilidade, mas também aumentar o custo de execução. O ponto de partida é **Trade-offs no Projeto de Linguagens**: Uma decisão de projeto pode beneficiar um critério e prejudicar outro. Os contrastes destacados são confiabilidade versus custo de execução, legibilidade versus capacidade de escrita e flexibilidade versus confiabilidade. A discussão avança para **Endereço e apelidos**: O endereço é a posição de memória associada à variável. Quando dois nomes acessam a mesma posição, eles são apelidos; isso pode prejudicar a legibilidade porque o programador precisa acompanhar referências alternativas. Outro elemento dessa análise é **Declaração e vinculação dinâmica de tipos**: Declaração explícita associa nomes a tipos por uma sentença; declaração implícita usa convenções. Na vinculação dinâmica de tipos, atribuições podem alterar o tipo associado, aumentando flexibilidade, mas dificultando a detecção antecipada de erros.

Fontes: slides-ch1, slide 3, paraphrase; slides-ch1, slide 5, paraphrase; slides-ch1, slide 7, paraphrase; slides-ch1, slide 14, paraphrase; slides-ch1, slide 18, paraphrase; slides-ch1, slide 27, paraphrase; slides-ch5, slide 11, paraphrase; slides-ch5, slide 19, paraphrase; slides-ch5, slide 21, paraphrase

Custo de uma linguagem: critérios e conexões

O ponto de partida é **Custo de uma linguagem**: O custo inclui treinamento, aprendizagem, escrita de programas, implementação e execução. Decisões como verificações em tempo de execução podem elevar a confiabilidade, mas também aumentar o custo de execução. A discussão avança para **Influências no Projeto de Linguagens**: O projeto de linguagens é influenciado pela arquitetura de computadores e pelas metodologias de projeto de software. A Arquitetura de von Neumann favoreceu o modelo imperativo, enquanto novas metodologias contribuíram para novos paradigmas. Outro elemento dessa análise é **Categorias de Linguagens**: As categorias apresentadas são imperativas, funcionais, lógicas e híbridas de marcação-programação. Elas diferem pela centralidade de variáveis e atribuições, aplicação de funções, regras declarativas ou combinação de marcação com ações de programação. Na prática, a análise passa por **Trade-offs no Projeto de Linguagens**: Uma decisão de projeto pode beneficiar um critério e prejudicar outro. Os contrastes destacados são confiabilidade versus custo de execução, legibilidade versus capacidade de escrita e flexibilidade versus confiabilidade. Ao mesmo tempo, o quadro se completa com **Métodos de Implementação e Ambientes de Programação**: Compilação traduz para linguagem de máquina; interpretação pura executa por meio de um interpretador; sistemas híbridos usam código intermediário; e JIT compila durante a execução. Ambientes de programação reúnem ferramentas de desenvolvimento. Por fim, esta parte se encerra com **Java, Lua e estratégias de implementação**: Java representa a combinação do paradigma imperativo com orientação a objetos e um sistema de execução amplamente disponível. Lua combina tipagem dinâmica, tabelas, funções de primeira classe, coleta de lixo e tradução para código intermediário interpretado.

Fontes: slides-ch1, slide 18, paraphrase; slides-ch1, slide 21, paraphrase; slides-ch1, slide 22, paraphrase; slides-ch1, slide 26, paraphrase; slides-ch1, slide 27, paraphrase; slides-ch1, slide 28, paraphrase; slides-ch1, slide 36, paraphrase; slides-ch1, slide 38, paraphrase; sebesta-11ed, p. 106, paraphrase; sebesta-11ed, p. 113, paraphrase; slides-ch1, slide 34, paraphrase

Evolução das principais linguagens de programação: critérios e conexões

O ponto de partida é **Evolução das principais linguagens de programação**: A evolução deve ser estudada pela relação entre o ambiente de criação, as motivações, as contribuições e a influência sobre linguagens posteriores, e não como uma sequência em que cada linguagem simplesmente substitui a anterior. A discussão avança para **Plankalkül**: Plankalkül foi concebida por Konrad Zuse para expressar computações, mas não foi implementada em sua época e teve sua descrição publicada muito depois. Sua relevância histórica decorre de antecipar recursos avançados, especialmente em estruturas de dados. Outro elemento dessa análise é **Pseudocódigos e interpretação inicial**: Sistemas iniciais como Short Code e Speedcoding simplificaram a programação por meio de interpretação e operações de nível mais alto, mas pagaram por isso com maior tempo de execução e limitações de recursos. Na prática, a análise passa por **Fortran e computação científica**: A evolução de Fortran incorporou recursos como declarações de tipo explícitas, estruturas de controle, passagem de subprogramas, vetores dinâmicos, registros, ponteiros, módulos e recursão, acompanhando novas necessidades de programação científica. Ao mesmo tempo, o quadro se completa com **Lisp, Scheme e programação funcional**: A linhagem de Lisp favorece computação baseada em funções e listas. Scheme destaca tamanho reduzido, escopo estático e funções como entidades de primeira classe, que podem ser atribuídas, passadas e retornadas. Por fim, esta parte se encerra com **Linguagens orientadas por domínio**: COBOL e Basic ilustram projetos guiados por públicos e domínios distintos: processamento comercial com notação próxima da linguagem natural e ampliação do acesso à programação para estudantes não especializados. O ponto de partida é **Linguagens dinâmicas: APL e SNOBOL**: APL e SNOBOL apresentam tipagem dinâmica e alocação dinâmica de armazenamento: a variável adquire tipo e armazenamento quando recebe um valor. A discussão avança para **Linhagens imperativa, lógica e orientada a objetos**: A revisão histórica contrasta linhagens baseadas em comandos e estado, programação baseada em regras e linguagens orientadas a objetos. Ada relaciona recursos de linguagem a sistemas críticos, e Smalltalk ocupa posição central no suporte completo à orientação a objetos. Outro elemento dessa análise é **Java, Lua e estratégias de implementação**: Java representa a combinação do paradigma imperativo com orientação a objetos e um sistema de execução amplamente disponível. Lua combina tipagem dinâmica, tabelas, funções de primeira classe, coleta de lixo e tradução para código intermediário interpretado. Na prática, a análise passa por **Linguagens híbridas de marcação-programação**: Uma linguagem híbrida de marcação-programação estende marcação com elementos capazes de especificar ações como controle de fluxo e computação. XSLT e JSP aparecem como exemplos históricos dessa categoria.

Fontes: sebesta-11ed, p. 50, paraphrase; exercises-ch2, p. 1, inference; sebesta-11ed, p. 52, paraphrase; exercises-ch2, p. 1, teacher_provided; sebesta-11ed, p. 55, paraphrase; sebesta-11ed, p. 59, paraphrase; sebesta-11ed, p. 65, paraphrase; sebesta-11ed, p. 73, paraphrase; sebesta-11ed, p. 77, paraphrase; sebesta-11ed, p. 85, paraphrase; exercises-ch2, p. 2, teacher_provided; sebesta-11ed, p. 92, paraphrase; sebesta-11ed, p. 98, paraphrase; sebesta-11ed, p. 106, paraphrase; sebesta-11ed, p. 113, paraphrase; slides-ch1, slide 34, paraphrase; sebesta-11ed, p. 116, paraphrase; exercises-ch2, p. 3, teacher_provided

Categorias de Linguagens: critérios e conexões

O ponto de partida é **Categorias de Linguagens**: As categorias apresentadas são imperativas, funcionais, lógicas e híbridas de marcação-programação. Elas diferem pela centralidade de variáveis e atribuições, aplicação de funções, regras declarativas ou combinação de marcação com ações de programação. A discussão avança para **Plankalkül**: Plankalkül foi concebida por Konrad Zuse para expressar computações, mas não foi implementada em sua época e teve sua descrição publicada muito depois. Sua relevância histórica decorre de antecipar recursos avançados, especialmente em estruturas de dados. Outro elemento dessa análise é **Pseudocódigos e interpretação inicial**: Sistemas iniciais como Short Code e Speedcoding simplificaram a programação por meio de interpretação e operações de nível mais alto, mas pagaram por isso com maior tempo de execução e limitações de recursos. Na prática, a análise passa por **Fortran e computação científica**: A evolução de Fortran incorporou recursos como declarações de tipo explícitas, estruturas de controle, passagem de subprogramas, vetores dinâmicos, registros, ponteiros, módulos e recursão, acompanhando novas necessidades de programação científica. Ao mesmo tempo, o quadro se completa com **Lisp, Scheme e programação funcional**: A linhagem de Lisp favorece computação baseada em funções e listas. Scheme destaca tamanho reduzido, escopo estático e funções como entidades de primeira classe, que podem ser atribuídas, passadas e retornadas. Por fim, esta parte se encerra com **Linguagens orientadas por domínio**: COBOL e Basic ilustram projetos guiados por públicos e domínios distintos: processamento comercial com notação próxima da linguagem natural e ampliação do acesso à programação para estudantes não especializados. O ponto de partida é **Linguagens dinâmicas: APL e SNOBOL**: APL e SNOBOL apresentam tipagem dinâmica e alocação dinâmica de armazenamento: a variável adquire tipo e armazenamento quando recebe um valor. A discussão avança para **Linguagens imperativa, lógica e orientada a objetos**: A revisão histórica contrasta linguagens baseadas em comandos e estado, programação baseada em regras e linguagens orientadas a objetos. Ada relaciona recursos de linguagem a sistemas críticos, e Smalltalk ocupa posição central no suporte completo à orientação a objetos. Outro elemento dessa análise é **Java, Lua e estratégias de implementação**: Java representa a combinação do paradigma imperativo com orientação a objetos e um sistema de execução amplamente disponível. Lua combina tipagem dinâmica, tabelas, funções de primeira classe, coleta de lixo e tradução para código intermediário interpretado. Na prática, a análise passa por **Linguagens híbridas de marcação-programação**: Uma linguagem híbrida de marcação-programação estende marcação com elementos capazes de especificar ações como controle de fluxo e computação. XSLT e JSP aparecem como exemplos históricos dessa categoria.

Fontes: slides-ch1, slide 26, paraphrase; sebesta-11ed, p. 52, paraphrase; exercises-ch2, p. 1, teacher_provided; sebesta-11ed, p. 55, paraphrase; sebesta-11ed, p. 59, paraphrase; sebesta-11ed, p. 65, paraphrase; sebesta-11ed, p. 73, paraphrase; sebesta-11ed, p. 77, paraphrase; sebesta-11ed, p. 85, paraphrase; exercises-ch2, p. 2, teacher_provided; sebesta-11ed, p. 92, paraphrase; sebesta-11ed, p. 98, paraphrase; sebesta-11ed, p. 106, paraphrase; sebesta-11ed, p. 113, paraphrase; slides-ch1, slide 34, paraphrase; sebesta-11ed, p. 116, paraphrase; exercises-ch2, p. 3, teacher_provided

Sintaxe e Semântica: critérios e conexões

O ponto de partida é **Sintaxe e Semântica**: Sintaxe é a forma ou estrutura das expressões, atribuições e unidades de programa; semântica é o significado dessas construções. Juntas, caracterizam a definição de uma linguagem de programação. A discussão avança para **Sentença, linguagem, lexema e token**: Sentença é uma cadeia de caracteres sobre um alfabeto; linguagem é um conjunto de sentenças; lexema é uma unidade sintática elementar; e token é uma categoria que reúne lexemas. Outro elemento dessa análise é **Reconhecedores e Geradores**: Um reconhecedor decide se uma cadeia pertence à linguagem. Um gerador descreve ou produz as sentenças da linguagem e oferece uma estrutura com a qual uma sentença pode ser comparada. Na prática, a análise passa por **Casamento de padrões e geração de tokens**: O analisador léxico encontra subcadeias no código-fonte, agrupa lexemas e atribui códigos internos. Um token pode representar vários lexemas, como diferentes nomes classificados como IDENT.

Fontes: slides-ch3, slide 3, paraphrase; slides-ch3, slide 4, paraphrase; slides-ch3, slide 6, paraphrase; slides-ch4, slide 7, paraphrase

Reconhecedores e Geradores: critérios e conexões

O ponto de partida é **Reconhecedores e Geradores**: Um reconhecedor decide se uma cadeia pertence à linguagem. Um gerador descreve ou produz as sentenças da linguagem e oferece uma estrutura com a qual uma sentença pode ser comparada. A discussão avança para **FNB e Gramáticas Livres de Contexto**: Gramáticas Livres de Contexto são geradores usados para descrever classes de linguagens. A Forma Normal de Backus foi criada para descrever a sintaxe de Algol 58 e é equivalente às gramáticas livres de contexto. Outro elemento dessa análise é **Terminais, não terminais e regras**: Na FNB, não terminais representam classes de estruturas sintáticas, enquanto terminais correspondem a tokens ou lexemas. Cada regra tem um não terminal à esquerda e terminais e/ou não terminais à direita. Na prática, a análise passa por **Recursão, derivação e forma sentencial**: Recursão permite descrever listas sintáticas. Derivação é a aplicação repetida de regras desde o símbolo inicial; cada cadeia intermediária é uma forma sentencial, e uma forma composta apenas por terminais é uma sentença. Ao mesmo tempo, o quadro se completa com **Árvore de Análise Sintática**: É a representação hierárquica de uma derivação, mostrando como símbolos não terminais são expandidos até produzir os terminais da sentença. Por fim, esta parte se encerra com **Ambiguidade em Gramáticas**: Uma gramática é ambígua quando uma mesma sentença pode produzir duas árvores de derivação diferentes. A organização das regras pode expressar precedência de operadores e eliminar uma ambiguidade. O ponto de partida é **FNB Extendida**: A FNBE usa colchetes para termos opcionais, parênteses e barras para alternativas e chaves para repetições de zero ou mais termos. A discussão avança para **Recursão à esquerda**: Gramáticas com recursão à esquerda direta ou indireta não podem ser usadas diretamente por um analisador descendente. A gramática deve ser transformada para substituir essa recursão por uma forma adequada.

Fontes: slides-ch3, slide 6, paraphrase; slides-ch3, slide 7, paraphrase; slides-ch3, slide 8, paraphrase; slides-ch3, slide 11, paraphrase; slides-ch3, slide 14, paraphrase; slides-ch3, slide 15, paraphrase; slides-ch3, slide 16, paraphrase; slides-ch3, slide 18, paraphrase; slides-ch3, slide 21, paraphrase; slides-ch4, slide 36, paraphrase

Ambiguidade em Gramáticas e Semântica Estática e Semântica Dinâmica

O ponto de partida é **Ambiguidade em Gramáticas**: Uma gramática é ambígua quando uma mesma sentença pode produzir duas árvores de derivação diferentes. A organização das regras pode expressar precedência de operadores e eliminar uma ambiguidade. A discussão avança para **Semântica Estática e Semântica Dinâmica**: A semântica estática trata restrições que uma GLC não descreve adequadamente e pode ser especificada por gramáticas de atributos. A semântica dinâmica pode ser descrita pelos métodos operacional, denotacional e axiomático.

Fontes: slides-ch3, slide 16, paraphrase; slides-ch3, slide 18, paraphrase; slides-ch3, slide 24, paraphrase; slides-ch3, slide 25, paraphrase; slides-ch3, slide 34, paraphrase; slides-ch3, slide 39, paraphrase; slides-ch3, slide 49, paraphrase

Métodos de Implementação e Ambientes de Programação: critérios e conexões

O ponto de partida é **Métodos de Implementação e Ambientes de Programação**: Compilação traduz para linguagem de máquina; interpretação pura executa por meio de um interpretador; sistemas híbridos usam código intermediário; e JIT compila durante a execução. Ambientes de programação reúnem ferramentas de desenvolvimento. A discussão avança para **Sentença, linguagem, lexema e token**: Sentença é uma cadeia de caracteres sobre um alfabeto; linguagem é um conjunto de sentenças; lexema é uma unidade sintática elementar; e token é uma categoria que reúne lexemas. Outro elemento dessa análise é **Reconhecedores e Geradores**: Um reconhecedor decide se uma cadeia pertence à linguagem. Um gerador descreve ou produz as sentenças da linguagem e oferece uma estrutura com a qual uma sentença pode ser comparada. Na prática, a análise passa por **Árvore de Análise Sintática**: É a representação hierárquica de uma derivação, mostrando como símbolos não terminais são expandidos até produzir os terminais da sentença. Ao mesmo tempo, o quadro se completa com **Análise Léxica e Análise Sintática**: O processamento sintático é dividido entre o analisador léxico, baseado em padrões e gramática regular, e o parser, baseado em GLC ou FNB e modelado como autômato com pilha. Por fim, esta parte se encerra com **Separação entre léxico e parser**: Separar análise léxica e análise sintática favorece simplicidade, permite otimizações específicas do léxico e isola aspectos menos portáteis. O ponto de partida é **Casamento de padrões e geração de tokens**: O analisador léxico encontra subcadeias no código-fonte, agrupa lexemas e atribui códigos internos. Um token pode representar vários lexemas, como diferentes nomes classificados como IDENT. A discussão avança para **Implementação do analisador léxico**: O analisador léxico pode ser construído com ferramentas a partir de descrições formais, com código baseado em diagramas de estado ou com tabelas montadas manualmente. Funções como getChar, addChar e lookup apoiam o reconhecimento. Outro elemento dessa análise é **Objetivos da análise sintática**: O parser deve localizar erros sintáticos, produzir diagnósticos, recuperar-se para continuar a análise e construir a árvore de análise sintática ou um caminho correspondente.

Fontes: slides-ch1, slide 28, paraphrase; slides-ch1, slide 36, paraphrase; slides-ch1, slide 38, paraphrase; slides-ch3, slide 4, paraphrase; slides-ch3, slide 6, paraphrase; slides-ch3, slide 15, paraphrase; slides-ch4, slide 4, paraphrase; slides-ch4, slide 6, paraphrase; slides-ch4, slide 7, paraphrase; slides-ch4, slide 9, paraphrase; slides-ch4, slide 15, paraphrase; slides-ch4, slide 18, paraphrase

Recursão, derivação e forma sentencial: critérios e conexões

O ponto de partida é **Recursão, derivação e forma sentencial**: Recursão permite descrever listas sintáticas. Derivação é a aplicação repetida de regras desde o símbolo inicial; cada cadeia intermediária é uma forma sentencial, e uma forma composta apenas por terminais é uma sentença. A discussão avança para **Árvore de Análise Sintática**: É a representação hierárquica de uma derivação, mostrando como símbolos não terminais são expandidos até produzir os terminais da sentença. Outro elemento dessa análise é **Ambiguidade em Gramáticas**: Uma gramática é ambígua quando uma mesma sentença pode produzir duas árvores de derivação diferentes. A organização das regras pode expressar precedência de operadores e eliminar uma ambiguidade. Na prática, a análise passa por **FNB Extendida**: A FNBE usa colchetes para termos opcionais, parênteses e barras para alternativas e chaves para repetições de zero ou mais termos. Ao mesmo tempo, o quadro se completa com **Objetivos da análise sintática**: O parser deve localizar erros sintáticos, produzir diagnósticos, recuperar-se para continuar a análise e construir a árvore de análise sintática ou um caminho correspondente. Por fim, esta parte se encerra com **Análise Sintática Descendente**: A análise descendente constrói a árvore a partir da raiz, usa derivação mais à esquerda e pode ser implementada por uma coleção de subprogramas, muitos deles recursivos. O ponto de partida é **Lookahead**: Quando um não terminal possui alternativas, o analisador descendente compara o próximo token com os primeiros tokens que cada regra pode gerar. A ausência de correspondência indica erro sintático. A discussão avança para **Recursão à esquerda**: Gramáticas com recursão à esquerda direta ou indireta não podem ser usadas diretamente por um analisador descendente. A gramática deve ser transformada para substituir essa recursão por uma forma adequada. Outro elemento dessa análise é **Análise Sintática Ascendente e redução**: A análise ascendente constrói a árvore a partir das folhas e procura uma subcadeia que corresponda ao lado direito de uma regra, reduzindo-a para recuperar a forma sentencial anterior da derivação mais à direita.

Fontes: slides-ch3, slide 11, paraphrase; slides-ch3, slide 14, paraphrase; slides-ch3, slide 15, paraphrase; slides-ch3, slide 16, paraphrase; slides-ch3, slide 18, paraphrase; slides-ch3, slide 21, paraphrase; slides-ch4, slide 18, paraphrase; slides-ch4, slide 19, paraphrase; slides-ch4, slide 26, paraphrase; slides-ch4, slide 31, paraphrase; slides-ch4, slide 36, paraphrase; slides-ch4, slide 22, paraphrase; slides-ch4, slide 42, paraphrase

Análise Sintática Ascendente e redução e Analisadores LR, ACTION e GOTO

O ponto de partida é **Análise Sintática Ascendente e redução**: A análise ascendente constrói a árvore a partir das folhas e procura uma subcadeia que corresponda ao lado direito de uma regra, reduzindo-a para recuperar a forma sentencial anterior da derivação mais à direita. A discussão avança para **Analisadores LR, ACTION e GOTO**: Analisadores LR são analisadores ascendentes normalmente construídos com ferramentas. A tabela ACTION determina ações com base no estado e no próximo terminal; GOTO determina o novo estado depois de uma redução usando não terminais.

Fontes: slides-ch4, slide 22, paraphrase; slides-ch4, slide 42, paraphrase; slides-ch4, slide 49, paraphrase; slides-ch4, slide 51, paraphrase

Linguagens dinâmicas: APL e SNOBOL: critérios e conexões

O ponto de partida é **Linguagens dinâmicas: APL e SNOBOL**: APL e SNOBOL apresentam tipagem dinâmica e alocação dinâmica de armazenamento: a variável adquire tipo e armazenamento quando recebe um valor. A discussão avança para **Nomes e palavras especiais**: O projeto de nomes considera distinção entre maiúsculas e minúsculas e tratamento das palavras especiais. Palavra-chave é especial em certos contextos; palavra reservada não pode ser usada como nome. Outro elemento dessa análise é **Variável e seus atributos**: Uma variável é uma abstração de uma célula de memória e pode ser caracterizada por nome, endereço, valor, tipo, tempo de vida e escopo. Na prática, a análise passa por **Endereço e apelidos**: O endereço é a posição de memória associada à variável. Quando dois nomes acessam a mesma posição, eles são apelidos; isso pode prejudicar a legibilidade porque o programador precisa acompanhar referências alternativas. Ao mesmo tempo, o quadro se completa com **Vinculação e tempo de vinculação**: Vinculação é uma associação entre atributo e entidade, como variável e tipo ou operação e símbolo. Tempo de vinculação é o momento em que a associação ocorre. Por fim, esta parte se encerra com **Vinculação estática e dinâmica**: A vinculação estática ocorre antes da execução e permanece inalterada; a dinâmica ocorre pela primeira vez durante a execução ou pode mudar enquanto o programa executa. O ponto de partida é **Declaração e vinculação dinâmica de tipos**: Declaração explícita associa nomes a tipos por uma sentença; declaração implícita usa convenções. Na vinculação dinâmica de tipos, atribuições podem alterar o tipo associado, aumentando flexibilidade, mas dificultando a detecção antecipada de erros. A discussão avança para **Alocação, desalocação e tempo de vida**: Alocação obtém uma célula de memória e desalocação a devolve. O tempo de vida de uma variável é o período em que ela permanece vinculada a uma posição específica de memória. Outro elemento dessa análise é **Categorias de variáveis por tempo de vida**: As categorias apresentadas são estáticas, dinâmicas da pilha, dinâmicas de heap e dinâmicas de heap implícito. Elas diferem pelo momento e mecanismo de alocação, flexibilidade, eficiência, suporte à recursão e possibilidade de detecção de erros. Na prática, a análise passa por **Escopo, tempo de vida e constantes nomeadas**: Escopo e tempo de vida são conceitos diferentes: um trata visibilidade textual e o outro vinculação à memória. Uma constante nomeada recebe uma única vinculação de valor e favorece legibilidade, modificabilidade e parametrização.

Fontes: sebesta-11ed, p. 85, paraphrase; exercises-ch2, p. 2, teacher_provided; slides-ch5, slide 4, paraphrase; slides-ch5, slide 8, paraphrase; slides-ch5, slide 9, paraphrase; slides-ch5, slide 11, paraphrase; slides-ch5, slide 13, paraphrase; slides-ch5, slide 14, paraphrase; slides-ch5, slide 17, paraphrase; slides-ch5, slide 19, paraphrase; slides-ch5, slide 21, paraphrase; slides-ch5, slide 22, paraphrase; slides-ch5, slide 23, paraphrase; slides-ch5, slide 24, paraphrase; slides-ch5, slide 25, paraphrase; slides-ch5, slide 26, paraphrase; slides-ch5, slide 41, paraphrase; slides-ch5, slide 45, paraphrase

Lisp, Scheme e programação funcional: critérios e conexões

O ponto de partida é **Lisp, Scheme e programação funcional**: A linhagem de Lisp favorece computação baseada em funções e listas. Scheme destaca tamanho reduzido, escopo estático e funções como entidades de primeira classe, que podem ser atribuídas, passadas e retornadas. A discussão avança para **Alocação, desalocação e tempo de vida**: Alocação obtém uma célula de memória e desalocação a devolve. O tempo de vida de uma variável é o período em que ela permanece vinculada a uma posição específica de memória. Outro elemento dessa análise é **Escopo estático, escopo dinâmico e ambiente de referenciamento**: Escopo é a faixa de sentenças em que uma variável é visível. No escopo estático, a declaração é encontrada pela organização textual e pelos ancestrais estáticos; no dinâmico, pela sequência de chamadas. O ambiente de referenciamento reúne as variáveis visíveis em uma sentença. Na prática, a análise passa por **Escopo, tempo de vida e constantes nomeadas**: Escopo e tempo de vida são conceitos diferentes: um trata visibilidade textual e o outro vinculação à memória. Uma constante nomeada recebe uma única vinculação de valor e favorece legibilidade, modificabilidade e parametrização.

Fontes: sebesta-11ed, p. 65, paraphrase; slides-ch5, slide 22, paraphrase; slides-ch5, slide 27, paraphrase; slides-ch5, slide 28, paraphrase; slides-ch5, slide 38, paraphrase; slides-ch5, slide 42, paraphrase; slides-ch5, slide 41, paraphrase; slides-ch5, slide 45, paraphrase

Razões para estudar conceitos de LP: critérios e conexões

O ponto de partida é **Razões para estudar conceitos de LP**: O estudo dos conceitos de linguagens de programação amplia a expressão de ideias, fundamenta a escolha de linguagens, facilita a aprendizagem de novas linguagens, melhora o uso das já conhecidas e ajuda a compreender a implementação. A discussão avança para **Critérios de Avaliação de Linguagem**: Legibilidade, capacidade de escrita, confiabilidade e custo são critérios usados para comparar a adequação de uma linguagem, sem pressupor que uma única linguagem maximize todos eles. Outro elemento dessa análise é **Confiabilidade e verificação de tipos**: Confiabilidade é o comportamento de acordo com as especificações. A verificação de tipos contribui para esse critério ao detectar usos incompatíveis, preferencialmente durante a compilação para reduzir o custo de correção. Na prática, a análise passa por **Influências no Projeto de Linguagens**: O projeto de linguagens é influenciado pela arquitetura de computadores e pelas metodologias de projeto de software. A Arquitetura de von Neumann favoreceu o modelo imperativo, enquanto novas metodologias contribuíram para novos paradigmas. Ao mesmo tempo, o quadro se completa com **Trade-offs no Projeto de Linguagens**: Uma decisão de projeto pode beneficiar um critério e prejudicar outro. Os contrastes destacados são confiabilidade versus custo de execução, legibilidade versus capacidade de escrita e flexibilidade versus confiabilidade. Por fim, esta parte se encerra com **Métodos de Implementação e Ambientes de Programação**: Compilação traduz para linguagem de máquina; interpretação pura executa por meio de um interpretador; sistemas híbridos usam código intermediário; e JIT compila durante a execução. Ambientes de programação reúnem ferramentas de desenvolvimento. O ponto de partida é **Evolução das principais linguagens de programação**: A evolução deve ser estudada pela relação entre o ambiente de criação, as motivações, as contribuições e a influência sobre linguagens posteriores, e não como uma sequência em que cada linguagem simplesmente substitui a anterior. A discussão avança para **Linhagens imperativa, lógica e orientada a objetos**: A revisão histórica contrasta linhagens baseadas em comandos e estado, programação baseada em regras e linguagens orientadas a objetos. Ada relaciona recursos de linguagem a sistemas críticos, e Smalltalk ocupa posição central no suporte completo à orientação a objetos. Outro elemento dessa análise é **Sintaxe e Semântica**: Sintaxe é a forma ou estrutura das expressões, atribuições e unidades de programa; semântica é o significado dessas construções. Juntas, caracterizam a definição de uma linguagem de programação. Na prática, a análise passa por **Semântica Estática e Semântica Dinâmica**: A semântica estática trata restrições que uma GLC não descreve adequadamente e pode ser especificada por gramáticas de atributos. A semântica dinâmica pode ser descrita pelos métodos operacional, denotacional e axiomático. Ao mesmo tempo, o quadro se completa com **Análise Léxica e Análise Sintática**: O processamento sintático é dividido entre o analisador léxico, baseado em padrões e gramática regular, e o parser, baseado em GLC ou FNB e modelado como autômato com pilha. Por fim, esta parte se encerra com **Nomes e palavras especiais**: O projeto de nomes considera distinção entre maiúsculas e minúsculas e tratamento das palavras especiais. Palavra-chave é especial em certos contextos; palavra reservada não pode ser usada como nome. O ponto de partida é **Vinculação e tempo de vinculação**: Vinculação é uma associação entre atributo e entidade, como variável e tipo ou operação e símbolo. Tempo de vinculação é o momento em que a associação ocorre. A discussão avança para **Categorias de variáveis por tempo de vida**: As categorias apresentadas são estáticas, dinâmicas da pilha, dinâmicas de heap e dinâmicas de heap implícito. Elas diferem pelo momento e mecanismo de alocação, flexibilidade, eficiência, suporte à recursão e possibilidade de detecção de erros. Outro elemento dessa análise é **Escopo estático, escopo dinâmico e ambiente de referenciamento**: Escopo é a faixa de sentenças em que uma variável é visível. No escopo estático, a declaração é encontrada pela organização textual e pelos ancestrais estáticos; no dinâmico, pela sequência de chamadas. O ambiente de referenciamento reúne as variáveis visíveis em uma sentença.

Fontes: slides-ch1, slide 3, paraphrase; slides-ch1, slide 5, paraphrase; slides-ch1, slide 14, paraphrase; slides-ch1, slide 21, paraphrase; slides-ch1, slide 22, paraphrase; slides-ch1, slide 27, paraphrase; slides-ch1, slide 28, paraphrase; slides-ch1, slide 36,

paraphrase; slides-ch1, slide 38, paraphrase; sebesta-11ed, p. 50, paraphrase; exercises-ch2, p. 1, inference; sebesta-11ed, p. 92, paraphrase; sebesta-11ed, p. 98, paraphrase; exercises-ch2, p. 2, teacher_provided; slides-ch3, slide 3, paraphrase; slides-ch3, slide 24, paraphrase; slides-ch3, slide 25, paraphrase; slides-ch3, slide 34, paraphrase; slides-ch3, slide 39, paraphrase; slides-ch3, slide 49, paraphrase; slides-ch4, slide 4, paraphrase; slides-ch5, slide 4, paraphrase; slides-ch5, slide 8, paraphrase; slides-ch5, slide 13, paraphrase; slides-ch5, slide 14, paraphrase; slides-ch5, slide 23, paraphrase; slides-ch5, slide 24, paraphrase; slides-ch5, slide 25, paraphrase; slides-ch5, slide 26, paraphrase; slides-ch5, slide 27, paraphrase; slides-ch5, slide 28, paraphrase; slides-ch5, slide 38, paraphrase; slides-ch5, slide 42, paraphrase

Exemplos

Comparação de linguagens para um sistema crítico

Exemplo autoral: Duda compara duas opções. A primeira verifica índices e tipos antecipadamente; a segunda prioriza flexibilidade. A escolha deve explicitar o trade-off entre confiabilidade, capacidade de escrita e custo.

Genealogia não é substituição

Exemplo autoral: Vicente monta uma rede em que uma linguagem influencia outra por sintaxe, paradigma ou implementação, sem concluir que a linguagem anterior deixou de ser útil em seu domínio.

Escolha por domínio

Exemplo autoral: para cálculo científico, regras declarativas e transformação de documentos, Vicente procura famílias diferentes em vez de escolher uma única linguagem para todos os problemas.

Lexema e token

Exemplo autoral: em "total = 7", "total" é um lexema classificado como IDENT, "=" pode ser classificado como ATRIB e "7" como NUM.

Derivação de uma lista

Exemplo autoral: usando <lista> → ident | ident, <lista>, a sentença "a, b, c" é obtida por aplicações recursivas até restarem somente terminais.

Ambiguidade de operadores

Exemplo autoral: se uma gramática permite duas árvores para "const - const / const", a precedência deve ser representada pela organização de <expr> e <term>.

Pipeline léxico-sintático

Exemplo autoral: o léxico entrega IDENT, ATRIB e NUM ao parser; o parser verifica se essa sequência corresponde a uma regra de atribuição e a posiciona na árvore.

Descendente versus ascendente

Exemplo autoral: Duda começa pelo símbolo inicial para ilustrar a análise descendente; depois começa pelos tokens e realiza reduções para ilustrar a ascendente.

Escopo não é tempo de vida

Exemplo autoral: uma variável static local pode permanecer vinculada à memória durante toda a execução, embora seu nome seja visível apenas dentro da função.

Vinculação dinâmica de tipos

Exemplo autoral: se "item" recebe primeiro uma lista e depois um número, a vinculação dinâmica permite a mudança, mas reduz a capacidade de o compilador antecipar certos erros.

Pontos de atenção

Ao aplicar esses critérios, vale evitar alguns atalhos de raciocínio: [Conteúdo autoral — Junior/Duda] Junior: "Uma linguagem mais nova sempre substitui a anterior." Duda corrige: a evolução também envolve

coexistência, influência e adequação a domínios. [Conteúdo autoral — Junior/Duda] Junior: "A linguagem com maior capacidade de escrita é automaticamente a mais legível." Duda corrige: esses critérios podem entrar em trade-off. [Conteúdo autoral — Junior/Vicente] Junior: "Compilação e interpretação são categorias sem combinações." Vicente conecta sistemas híbridos e JIT, que combinam tradução intermediária e execução. [Conteúdo autoral — Junior/Duda] Junior: "Token e lexema são sinônimos." Duda corrige: lexema é a ocorrência no código; token é sua categoria ou código interno. [Conteúdo autoral — Junior/Duda] Junior: "Toda forma sentencial é uma sentença." Duda corrige: somente a forma sentencial composta exclusivamente por terminais é uma sentença. [Conteúdo autoral — Junior/Vicente] Junior: "Duas derivações diferentes sempre tornam a gramática ambígua." Vicente corrige: a condição apresentada é a existência de duas árvores diferentes para a mesma sentença. [Conteúdo autoral — Junior/Duda] Junior: "O analisador léxico constrói sozinho a árvore sintática." Duda corrige: o léxico entrega tokens; o parser verifica a estrutura e produz a árvore ou seu caminho. [Conteúdo autoral — Junior/Vicente] Junior: "Análise descendente começa nas folhas." Vicente corrige: ela começa na raiz; a análise ascendente começa nas folhas. [Conteúdo autoral — Junior/Duda] Junior: "Recursão à esquerda é conveniente para qualquer parser recursivo." Duda corrige: ela impede o uso direto de análise descendente e deve ser removida. [Conteúdo autoral — Junior/Duda] Junior: "Escopo e tempo de vida são a mesma coisa." Duda corrige: escopo trata visibilidade; tempo de vida trata vinculação a uma posição de memória. [Conteúdo autoral — Junior/Vicente] Junior: "Vinculação dinâmica sempre melhora a linguagem." Vicente corrige: ela amplia flexibilidade, mas pode aumentar custo e dificultar a detecção antecipada de erros. [Conteúdo autoral — Junior/Duda] Junior: "Palavra-chave e palavra reservada são necessariamente iguais." Duda corrige: palavra-chave é especial em certos contextos; palavra reservada não pode ser usada como nome.

Bibliografia

Capítulo 1 — Preliminares. Material de aula em apresentação.

Lista autoral de exercícios do Capítulo 2 fornecida pelo professor.

Capítulo 3 — Descrição da Sintaxe e da Semântica. Material de aula em apresentação.

Capítulo 4 — Análise Léxica e Sintática. Material de aula em apresentação.

Capítulo 5 — Nomes, Vinculações e Escopo. Material de aula em apresentação.

SEBESTA, Robert W. Conceitos de linguagens de programação. Tradução de João Eduardo Nóbrega Tortello. 11. ed. Porto Alegre: Bookman, 2018.

Este resumo privilegia paráfrases e articula os conceitos para leitura contínua. Consulte as fontes originais para contexto e aprofundamento.